Sistemas de Informação	PROFESSOR: Diego Cardoso Borda Castro	
	CURSO: Sistema de Informação	
	DISCIPLINA: Gerência de Configuração	
	PERÍODO: 7º	DATA: 29 / 09 /2026
	ALUNO:	NOTA:

Você faz parte da equipe que desenvolve a AgendaSaúde, uma API REST em Node.js 20 com Express para agendamento de consultas de uma clínica, que utiliza o PostgreSQL como banco de dados. O projeto está versionado em Git seguindo o Git Flow, será empacotado com Docker e entregue em produção por uma pipeline de CI/CD no GitHub Actions.

1) A imagem abaixo mostra a situação atual do repositório. Escreva, na ordem, os comandos necessários para resolver cada item.

```
~/Documentos/git/agenda-api
→ git log --oneline --graph --all
* ae21da6 (origin/develop, develop) adiciona migration node-pg-migrate da tabela paciente
| * 649023b (HEAD → feature/agendamento) adiciona validacao de horario da consulta
| * 563d834 cria ConsultaService e ConsultaController
| * bd6913c cria entidade Consulta e ConsultaRepository
|/
* 051d08a cria entidade Paciente
* 265417a atualiza package.json para Node 20
* c7e0e44 (tag: v1.0.0, origin/main, main) configura projeto Express com Postgres
* 7a91356 commit inicial
~/Documentos/git/agenda-api
→ git status
No ramo feature/agendamento
Mudanças não preparadas para commit:
        modificado:    src/consulta/consultaService.js

Arquivos não monitorados:
        test/
~/Documentos/git/agenda-api
→ git log --stat --oneline -n 2
649023b adiciona validacao de horario da consulta
 src/consulta/consultaService.js | 5 +++++
 1 file changed, 5 insertions(+)
563d834 cria ConsultaService e ConsultaController
 .env                            | 3 +++
 src/consulta/consultaController.js | 4 ++++
 src/consulta/consultaService.js    | 4 ++++
 3 files changed, 11 insertions(+)
~/Documentos/git/agenda-api
→
```

a) Guarde as alterações que ainda não foram commitadas (o consultaService.js modificado e o arquivo novo da pasta test/) sem perdê-las. Elas serão usadas no item c. **[0,5 ponto]**

b) O commit 563d834 enviou o arquivo .env com a senha do banco. Refaça os dois últimos commits da feature/agendamento sem o .env. Os outros arquivos e as mensagens dos dois commits devem ser mantidos. **[1,0 ponto]**

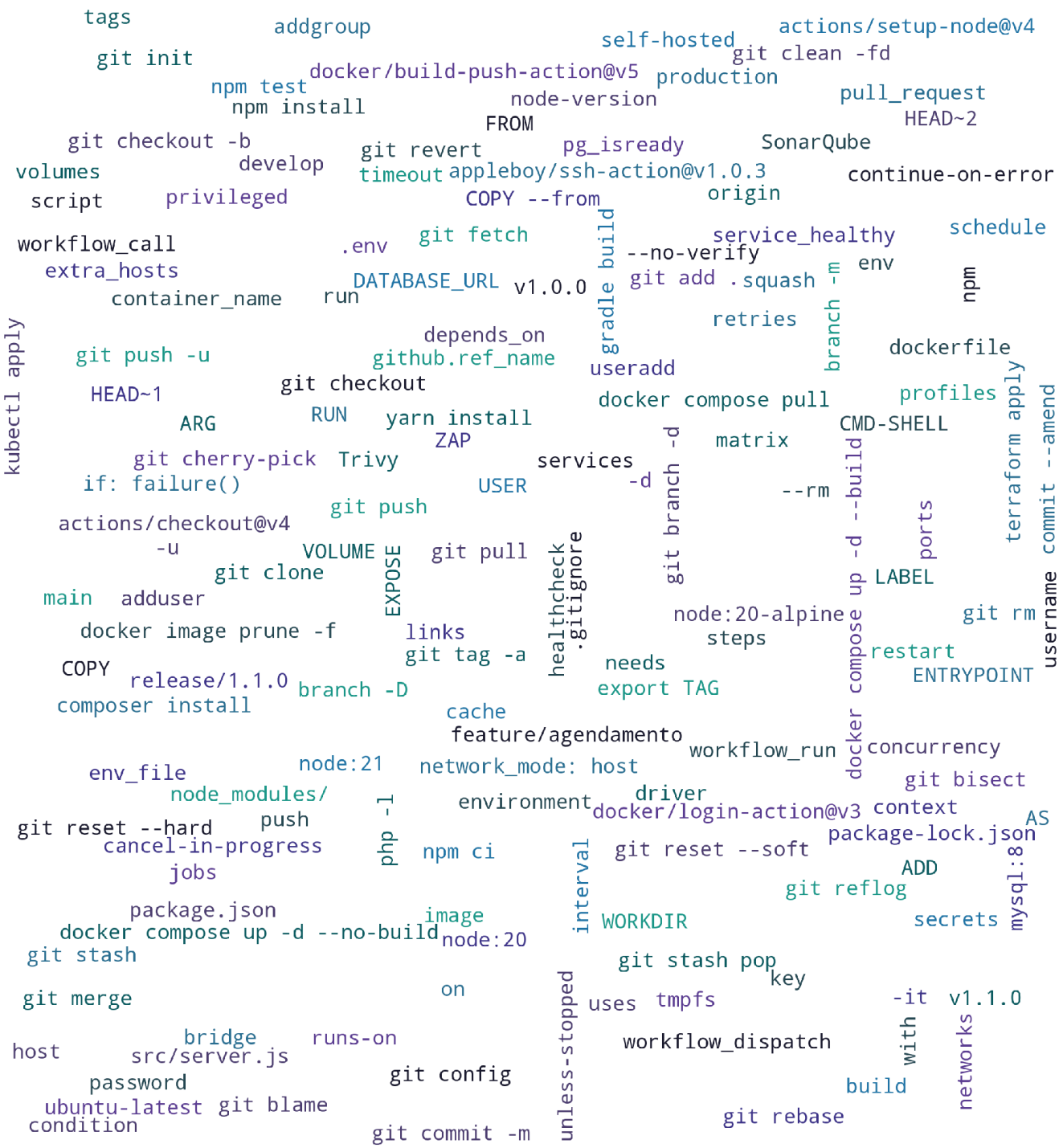
c) Atualize a feature/agendamento com o commit ae21da6 da develop utilizando rebase. Depois, recupere as alterações guardadas no item a, faça um commit com elas e envie a branch para o repositório remoto. **[0,5 ponto]**

2) Com o código já integrado, a AgendaSaúde precisa ser conteinerizada. Siga as instruções e detalhamentos do projeto para criar o **Dockerfile**, o **.dockerignore** e o **docker compose** necessários. **[3,0 pontos]**

- **a)** O projeto é um Node.js com npm: o package.json fica na raiz e o código-fonte na pasta src. O comando npm ci instala as dependências. A aplicação roda na porta 8080.
- **b)** Passe a versão do node como argumento.
- **c)** O .env preservado na Questão 1 está na raiz do projeto com as variáveis POSTGRES_DB, POSTGRES_USER e POSTGRES_PASSWORD. Acrescente a ele a variável DOCKERHUB_USERNAME com o seu usuário do Docker Hub. Nenhuma credencial pode ficar escrita diretamente no compose.
- **d)** Serviço do banco:
  - **i)** Use a imagem postgres:17-alpine, recebendo as variáveis do .env.
  - **ii)** Crie um healthcheck utilizando o comando pg_isready.
- **e)** Serviço da aplicação:
  - **i)** Deve ser construída a partir do Dockerfile e a imagem gerada deve se chamar ${DOCKERHUB_USERNAME}/agenda-api:${TAG:-latest}. Esse mesmo compose será usado no servidor de produção na Questão 3.
  - **ii)** Deve ficar acessível na porta 8080 da máquina host.
  - **iii)** Recebe a conexão com o banco pela variável DATABASE_URL. Ela segue o formato postgresql://USUARIO:SENHA@HOST:5432/BANCO. Não utilize IP fixo.
  - **iv)** O serviço da aplicação só pode subir se o PostgreSQL estiver saudável, e não apenas iniciado.

3) A equipe decidiu automatizar a entrega da AgendaSaúde com o GitHub Actions. A pipeline deve ser executada sempre que uma tag de versão (v*) for enviada para o repositório, como a v1.0.0 demonstrada na Questão 1, ou manualmente pela aba Actions. O servidor de produção é um Linux que já tem o docker e o compose instalados e, no diretório /opt/agenda-api, já possui o compose da Questão 2 e um .env de produção. **[3,0 pontos]**

- **a)** Secrets configurados no GitHub
  - **i)** DOCKERHUB_USERNAME, DOCKERHUB_TOKEN
  - **ii)** SSH_HOST, SSH_USER, SSH_PRIVATE_KEY
- **b)** Regras gerais
  - **i)** Nunca podem rodar dois deploys ao mesmo tempo.
  - **ii)** Os jobs são sequenciais: um job só começa se o anterior terminar com sucesso.
- **c)** Fluxo da pipeline esperado
  - **i)** Faz o build e roda os testes do Node.js
    1. use o actions/checkout@v4
    2. actions/setup-node@v4 com Node 20 e cache do npm
    3. Os testes de integração precisam de um PostgreSQL (postgres:17-alpine) rodando junto com o job. Passe a conexão para a aplicação pela variável DATABASE_URL.
    4. Comando: npm i && npm test
  - **ii)** Faz build da imagem e envia para o Docker Hub
    1. docker/login-action@v3
       - (a) username
       - (b) password
    2. docker/build-push-action@v5
  - **iii)** Acessa o servidor via SSH e atualiza a aplicação com docker compose
    1. appleboy/ssh-action@v1.0.3
       - (a) host
       - (b) username
       - (c) key
       - (d) script
         - (i) Entra em /opt/agenda-api, define a variável TAG com a versão que está sendo implantada, baixa a nova imagem da aplicação e recria o container sem fazer build no servidor. Por fim, remove as imagens antigas.


tags
addgroup
actions/setup-node@v4
self-hosted
git init
git clean -fd
docker/build-push-action@v5
production
npm test
npm install
node-version
pull_request
FROM
HEAD~2
git checkout -b
git revert
pg_isready
SonarQube
volumes
develop
timeout
appleboy/ssh-action@v1.0.3
continue-on-error
script
privileged
COPY --from
origin
workflow_call
.env
git fetch
service_healthy
schedule
--no-verify
extra_hosts
DATABASE_URL
v1.0.0
git add .squash -m
env
container_name
run
npm
retries
depends_on
dockerfile
git push -u
github.ref_name
useradd
HEAD~1
git checkout
docker compose pull
profiles
ARG
RUN
yarn install
CMD-SHELL
ZAP
matrix
git cherry-pick
Trivy
services
if: failure()
USER
-d
--rm
git push
actions/checkout@v4
-u
VOLUME
git pull
git clone
LABEL
main
adduser
node:20-alpine
git rm
docker image prune -f
links
steps
git tag -a
restart
COPY
release/1.1.0
needs
ENTRYPOINT
branch -D
export TAG
composer install
cache
feature/agendamento
workflow_run
concurrency
env_file
node:21
network_mode: host
git bisect
node_modules/
environment
driver
context
git reset --hard
push
docker/login-action@v3
AS
cancel-in-progress
npm ci
git reset --soft
package-lock.json
jobs
ADD
git reflog
package.json
image
WORKDIR
secrets
docker compose up -d --no-build
node:20
git stash
git stash pop
key
git merge
on
uses
tmpfs
-it
v1.1.0
bridge
runs-on
workflow_dispatch
host
src/server.js
password
git config
build
ubuntu-latest
git blame
condition
git commit -m
git rebase
kubectl apply
gradle build
branch -m
git branch -d
docker compose up -d --build
ports
terraform apply
commit --amend
username
healthcheck
.gitignore
EXPOSE
php -l
interval
unless-stopped
mysql:8
networks
with

Boa prova!